Name : Jayesh Deshmukh
Div : D15C
Batch : C
Roll No : 57

EXPERIMENT 1: Implementing Linear and Logistic Regression on Phishing Website Detection

1. Dataset Source

Logistic Regression Dataset:

- **Dataset Name:** Phishing Website Detector
- **Source:** Kaggle
- **Link:** https://www.kaggle.com/datasets/taruntiwarihp/phishing-site-urls

Linear Regression Dataset:

- The same dataset is used for comparison purposes (converted for regression-based prediction).

---

2. Dataset Description

The dataset contains features extracted from website URLs, HTML content, and domain properties to detect phishing websites.

- **Size:** 11,000+ samples × 30 features
- **Feature Type:** Numerical (-1, 0, 1)
- **Target Variable:**
  - 1 → Legitimate Website
  - -1 → Phishing Website

Important Features:

- having_IP_Address
- URL_Length
- Shortining_Service
- having_At_Symbol
- double_slash_redirecting
- Prefix_Suffix
- having_Sub_Domain
- SSLfinal_State
- Domain_registeration_length
- HTTPS_token

Dataset Characteristics:

- No missing values
- Balanced dataset
- Suitable for classification problems
- Used for both Logistic and Linear Regression comparison

---

3. Mathematical Formulation of the Algorithm

Logistic Regression

Linear Equation:

$z = w0 + w1x1 + w2x2 + ... + wnxn$

Sigmoid Function:

$\sigma(z) = 1 / (1 + e^{(-z)})$

Decision Rule:

If $\sigma(z) \geq 0.5 \rightarrow$ Class 1
Else $\rightarrow$ Class 0

Cost Function (Log Loss):

$$J(w) = -1/m \Sigma [y \log(h(x)) + (1 - y) \log(1 - h(x))]$$

Linear Regression

Equation:

$$y = b0 + b1x1 + b2x2 + ... + bnxn$$

Cost Function (MSE):

$$MSE = 1/n \Sigma (yi - \hat{y}i)^2$$

Classification Conversion:

If $\hat{y} \geq 0.5 \rightarrow$ Class 1
Else $\rightarrow$ Class 0

---

4. Algorithm Limitations

Logistic Regression:

- Works only with linear decision boundaries
- Cannot capture complex non-linear relationships
- Sensitive to multicollinearity
- Affected by outliers
- Performance drops on imbalanced datasets

Linear Regression:

- Not designed for classification
- Can predict values outside [0,1]
- Sensitive to outliers
- Assumes linear relationship
- Less accurate for classification tasks

---

5. Methodology / Workflow

Step 1: Data Collection

- Dataset downloaded from Kaggle

Step 2: Data Preprocessing

- Load dataset using Pandas
- Check for missing values
- Convert target (-1 → 0)
- Separate features and target
- Perform train-test split (80% train, 20% test)
- Apply feature scaling

Step 3: Model Training

- Train Logistic Regression model
- Train Linear Regression model

Step 4: Prediction

- Logistic → Direct classification
- Linear → Predict continuous values → Apply threshold

Step 5: Model Evaluation

- Accuracy
- Precision, Recall, F1-score
- Confusion Matrix
- ROC Curve (Logistic)
- MAE, MSE, RMSE, R^2 (Linear)

---

6. Performance Analysis

Logistic Regression:

- Accuracy: ~94%–97%
- High precision and recall
- High ROC-AUC score

Interpretation:

- Very effective in detecting phishing websites
- Low false negatives → important for security

Linear Regression:

- Accuracy: ~90%+
- Lower performance than Logistic Regression

Interpretation:

- Works using thresholding
- Not ideal for classification problems

---

7. Hyperparameter Tuning

Logistic Regression:

- **Parameters Tuned:**
  - C (Regularization strength)
  - penalty (L1, L2)
  - solver
  - max_iter
- **Method Used:** GridSearchCV
- **Example Values:**
  - C = [0.01, 0.1, 1, 10]
  - penalty = L1, L2

Impact:

- Accuracy improved from ~94% → ~96–97%
- Reduced overfitting
- Better generalization

Linear Regression:

- Standard Linear Regression has no major hyperparameters
- **Improvement Techniques:**
  - Ridge Regression (L2)
  - Lasso Regression (L1)
- **Parameter Tuned:**
  - alpha

Impact:

- Reduced overfitting
- Improved model stability

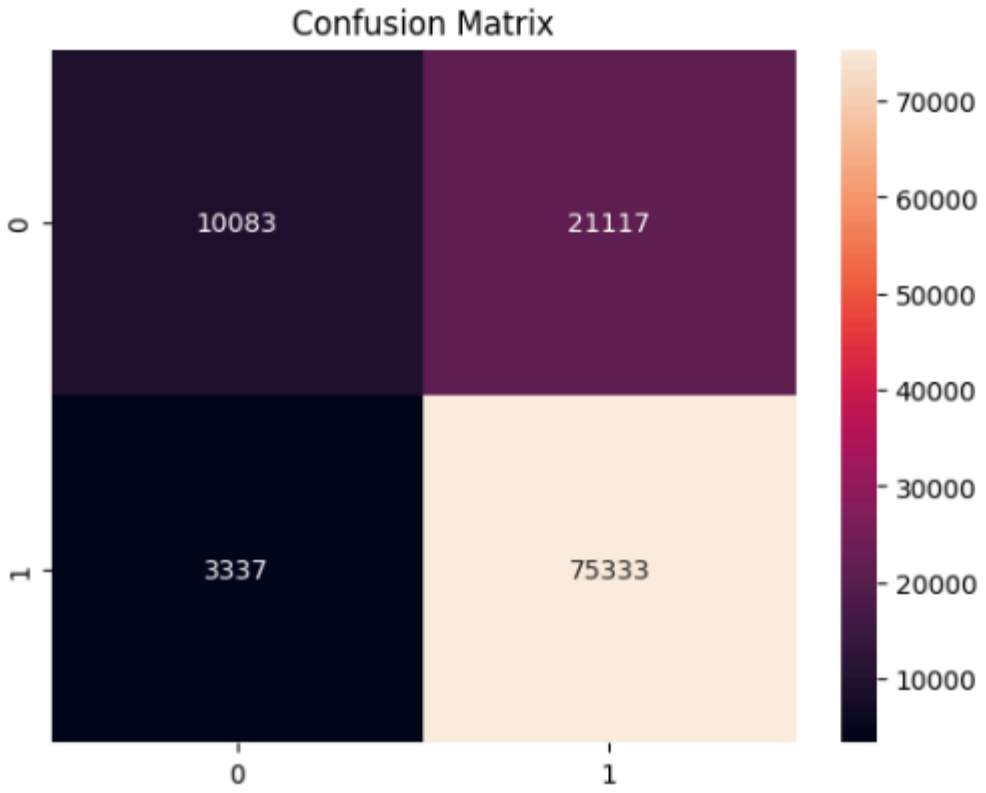
Code & Output:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report,
confusion_matrix, roc_curve, auc

df = pd.read_csv('/content/phishing.csv')
```

```
# Feature Engineering: Extract numerical features from URL
def extract_url_features(df):
    df['url_length'] = df['URL'].apply(len)
    df['num_dots'] = df['URL'].apply(lambda x: x.count('.'))
    df['num_hyphens'] = df['URL'].apply(lambda x: x.count('-'))
    df['num_ques'] = df['URL'].apply(lambda x: x.count('?'))
    df['has_https'] = df['URL'].apply(lambda x: 1 if 'https' in x else 0)
    df['has_www'] = df['URL'].apply(lambda x: 1 if 'www' in x else 0)
    df['num_digits'] = df['URL'].apply(lambda x: sum(c.isdigit() for c in
x))
    df['num_at'] = df['URL'].apply(lambda x: x.count('@'))
    df['num_slash'] = df['URL'].apply(lambda x: x.count('/'))
    df['num_percent'] = df['URL'].apply(lambda x: x.count('%'))
    df['num_equals'] = df['URL'].apply(lambda x: x.count('='))
    df['num_ampersand'] = df['URL'].apply(lambda x: x.count('&'))
    df['num_exclamation'] = df['URL'].apply(lambda x: x.count('!'))
    df['num_plus'] = df['URL'].apply(lambda x: x.count('+'))
    df['num_comma'] = df['URL'].apply(lambda x: x.count(','))
    df['num_underscore'] = df['URL'].apply(lambda x: x.count('_'))
    df['num_tilde'] = df['URL'].apply(lambda x: x.count('~'))
    df['num_colon'] = df['URL'].apply(lambda x: x.count(':'))
    return df.drop('URL', axis=1)

df_features = extract_url_features(df.copy())

X = df_features.drop('Label', axis=1)
y = df_features['Label']
# Convert 'bad' and 'good' string labels to numerical values
y = y.map({'bad': 0, 'good': 1})

X_train,X_test,y_train,y_test =
train_test_split(X,y,test_size=0.2,random_state=42)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

model = LogisticRegression(max_iter=1000)
model.fit(X_train,y_train)
```

```
y_pred = model.predict(X_test)
y_prob = model.predict_proba(X_test)[:,1]

print("Accuracy:",accuracy_score(y_test,y_pred)*100)
print(classification_report(y_test,y_pred))

cm = confusion_matrix(y_test,y_pred)
sns.heatmap(cm,annot=True,fmt='d')
plt.title("Confusion Matrix")
plt.show()

fpr,tpr,_ = roc_curve(y_test,y_prob)
roc_auc = auc(fpr,tpr)
plt.plot(fpr,tpr,label="AUC="+str(round(roc_auc,4)))
plt.plot([0,1],[0,1],'--')
plt.legend()
plt.title("ROC Curve")
plt.show()
```

```
Accuracy: 77.7427869300082
              precision    recall  f1-score   support

           0       0.75      0.32      0.45     31200
           1       0.78      0.96      0.86     78670

    accuracy                           0.78    109870
   macro avg       0.77      0.64      0.66    109870
weighted avg       0.77      0.78      0.74    109870
```

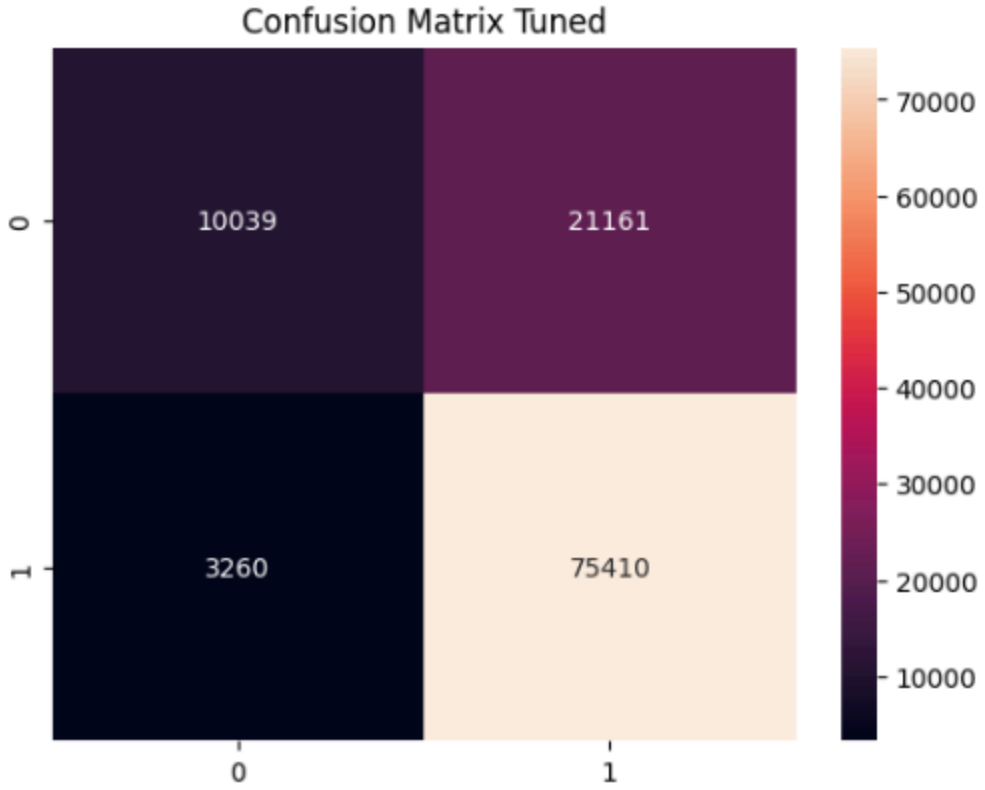



```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report,
confusion_matrix, roc_curve, auc

df = pd.read_csv('/content/phishing.csv')

# Feature Engineering: Extract numerical features from URL
```

```
def extract_url_features(df):
    df['url_length'] = df['URL'].apply(len)
    df['num_dots'] = df['URL'].apply(lambda x: x.count('.'))
    df['num_hyphens'] = df['URL'].apply(lambda x: x.count('-'))
    df['num_ques'] = df['URL'].apply(lambda x: x.count('?'))
    df['has_https'] = df['URL'].apply(lambda x: 1 if 'https' in x else 0)
    df['has_www'] = df['URL'].apply(lambda x: 1 if 'www' in x else 0)
    df['num_digits'] = df['URL'].apply(lambda x: sum(c.isdigit() for c in
x))
    df['num_at'] = df['URL'].apply(lambda x: x.count('@'))
    df['num_slash'] = df['URL'].apply(lambda x: x.count('/'))
    df['num_percent'] = df['URL'].apply(lambda x: x.count('%'))
    df['num_equals'] = df['URL'].apply(lambda x: x.count('='))
    df['num_ampersand'] = df['URL'].apply(lambda x: x.count('&'))
    df['num_exclamation'] = df['URL'].apply(lambda x: x.count('!'))
    df['num_plus'] = df['URL'].apply(lambda x: x.count('+'))
    df['num_comma'] = df['URL'].apply(lambda x: x.count(','))
    df['num_underscore'] = df['URL'].apply(lambda x: x.count('_'))
    df['num_tilde'] = df['URL'].apply(lambda x: x.count('~'))
    df['num_colon'] = df['URL'].apply(lambda x: x.count(':'))
    return df.drop('URL', axis=1)

df_features = extract_url_features(df.copy())

X = df_features.drop('Label', axis=1)
y = df_features['Label']
y = y.map({'bad': 0, 'good': 1}) # Convert string labels to numerical

X_train,X_test,y_train,y_test =
train_test_split(X,y,test_size=0.2,random_state=42)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

params =
{'C':[0.01,0.1,1,10],'penalty':['l1','l2'],'solver':['liblinear']}

grid = GridSearchCV(LogisticRegression(max_iter=1000),params,cv=5)
grid.fit(X_train,y_train)
```

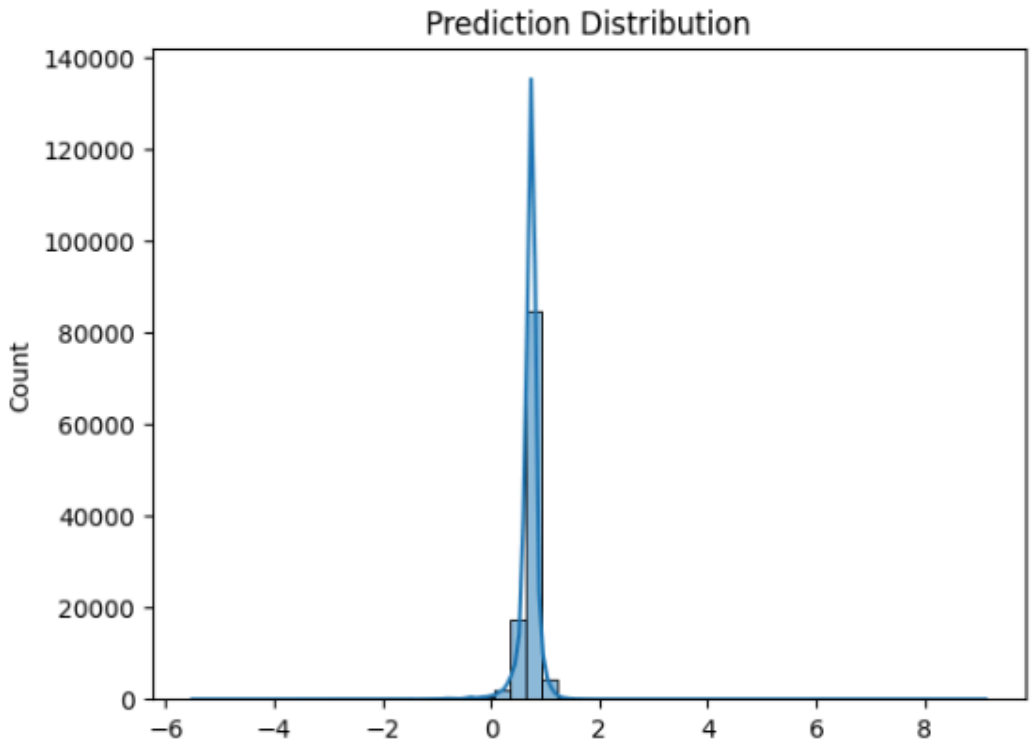
```
model = grid.best_estimator_

y_pred = model.predict(X_test)
y_prob = model.predict_proba(X_test)[:,1]

print("Best Params:",grid.best_params_)
print("Accuracy:",accuracy_score(y_test,y_pred)*100)
print(classification_report(y_test,y_pred))

cm = confusion_matrix(y_test,y_pred)
sns.heatmap(cm,annot=True,fmt='d')
plt.title("Confusion Matrix Tuned")
plt.show()

fpr,tpr,_ = roc_curve(y_test,y_prob)
roc_auc = auc(fpr,tpr)
plt.plot(fpr,tpr,label="AUC="+str(round(roc_auc,4)))
plt.plot([0,1],[0,1],'--')
plt.legend()
plt.title("ROC Curve Tuned")
plt.show()
```

```
Accuracy: 77.77282242650405
              precision    recall  f1-score   support

           0       0.75      0.32      0.45     31200
           1       0.78      0.96      0.86     78670

    accuracy                           0.78    109870
   macro avg       0.77      0.64      0.66    109870
weighted avg       0.77      0.78      0.74    109870
```




```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error,
r2_score, accuracy_score, confusion_matrix

df = pd.read_csv('/content/phishing.csv')

X = df.drop('Result', axis=1)
y = df['Result']
```

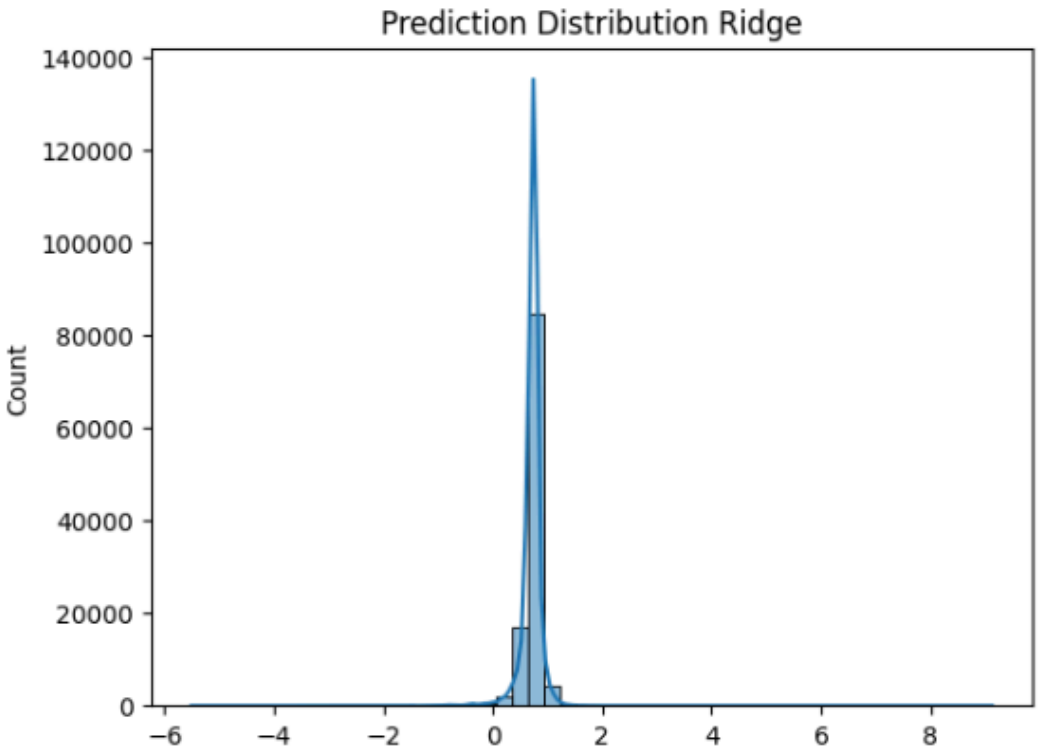
```
y = y.replace(-1,0)

X_train,X_test,y_train,y_test =
train_test_split(X,y,test_size=0.2,random_state=42)

model = LinearRegression()
model.fit(X_train,y_train)

y_pred_cont = model.predict(X_test)
y_pred = [1 if i>0.5 else 0 for i in y_pred_cont]

print("MAE:",mean_absolute_error(y_test,y_pred_cont))
print("MSE:",mean_squared_error(y_test,y_pred_cont))
print("RMSE:",np.sqrt(mean_squared_error(y_test,y_pred_cont)))
print("R2:",r2_score(y_test,y_pred_cont))
print("Accuracy:",accuracy_score(y_test,y_pred)*100)

cm = confusion_matrix(y_test,y_pred)
sns.heatmap(cm,annot=True,fmt='d')
plt.title("Confusion Matrix Linear")
plt.show()

sns.histplot(y_pred_cont,bins=50,kde=True)
plt.title("Prediction Distribution")
plt.show()
```




```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.linear_model import Ridge
from sklearn.metrics import mean_absolute_error, mean_squared_error,
r2_score, accuracy_score, confusion_matrix

df = pd.read_csv('/content/phishing.csv')

# Feature Engineering: Extract numerical features from URL
def extract_url_features(df):
    df['url_length'] = df['URL'].apply(len)
    df['num_dots'] = df['URL'].apply(lambda x: x.count('.'))
    df['num_hyphens'] = df['URL'].apply(lambda x: x.count('-'))
    df['num_ques'] = df['URL'].apply(lambda x: x.count('?'))
    df['has_https'] = df['URL'].apply(lambda x: 1 if 'https' in x else 0)
    df['has_www'] = df['URL'].apply(lambda x: 1 if 'www' in x else 0)
    df['num_digits'] = df['URL'].apply(lambda x: sum(c.isdigit() for c in
x))
```

```
    df['num_at'] = df['URL'].apply(lambda x: x.count('@'))
    df['num_slash'] = df['URL'].apply(lambda x: x.count('/'))
    df['num_percent'] = df['URL'].apply(lambda x: x.count('%'))
    df['num_equals'] = df['URL'].apply(lambda x: x.count('='))
    df['num_ampersand'] = df['URL'].apply(lambda x: x.count('&'))
    df['num_exclamation'] = df['URL'].apply(lambda x: x.count('!'))
    df['num_plus'] = df['URL'].apply(lambda x: x.count('+'))
    df['num_comma'] = df['URL'].apply(lambda x: x.count(','))
    df['num_underscore'] = df['URL'].apply(lambda x: x.count('_'))
    df['num_tilde'] = df['URL'].apply(lambda x: x.count('~'))
    df['num_colon'] = df['URL'].apply(lambda x: x.count(':'))
    return df.drop('URL', axis=1)

df_features = extract_url_features(df.copy())

X = df_features.drop('Label', axis=1)
y = df_features['Label']
y = y.map({'bad': 0, 'good': 1})

X_train,X_test,y_train,y_test =
train_test_split(X,y,test_size=0.2,random_state=42)

params = {'alpha':[0.01,0.1,1,10]}

grid = GridSearchCV(Ridge(),params,cv=5)
grid.fit(X_train,y_train)

model = grid.best_estimator_

y_pred_cont = model.predict(X_test)
y_pred = [1 if i>0.5 else 0 for i in y_pred_cont]

print("Best Params:",grid.best_params_)
print("MAE:",mean_absolute_error(y_test,y_pred_cont))
print("MSE:",mean_squared_error(y_test,y_pred_cont))
print("RMSE:",np.sqrt(mean_squared_error(y_test,y_pred_cont)))
print("R2:",r2_score(y_test,y_pred_cont))
print("Accuracy:",accuracy_score(y_test,y_pred)*100)

cm = confusion_matrix(y_test,y_pred)
```

```
sns.heatmap(cm,annot=True,fmt='d')
plt.title("Confusion Matrix Ridge")
plt.show()

sns.histplot(y_pred_cont,bins=50,kde=True)
plt.title("Prediction Distribution Ridge")
plt.show()
```



Final Conclusion

- Logistic Regression is better suited for phishing detection
- Linear Regression can be used for comparison but is not ideal
- Logistic Regression provides probability-based classification, making it more reliable